



UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Ingeniería

Estructuras de Datos

Quiz 03 – Dynamic Arrays and Amortized Analysis

Docente: Sebastián Yepes García **Semestre:** 2026-2
Fecha: 24 de septiembre de 2026 **Puntaje:** 5.0
Duración: 10 horas de preparación + 12 minutos de sustentación

Nombre:	_____
Documento:	_____
Grupo:	_____

Instrucciones generales.

El objetivo de esta actividad no es únicamente obtener una implementación funcional de *Snake*. Se evaluará la capacidad para **diseñar una estructura de datos**, establecer sus invariantes, analizar su comportamiento y justificar formalmente su eficiencia.

- La solución deberá implementarse en **Java 16 o superior** (se utilizan **records**).
- No se permite utilizar `ArrayList`, `Vector`, `LinkedList`, `Stack`, `Queue`, `Deque` ni estructuras dinámicas equivalentes de la biblioteca estándar.
- Puede utilizarse un arreglo nativo de Java. Dado que `new T[n]` no compila en Java, se permite crear el arreglo como `(T[]) new Object[n]` (con `@SuppressWarnings("unchecked")`) o trabajar directamente con `Object[]`.
- No se permite utilizar `Arrays.copyOf()` ni `System.arraycopy()` para implementar el redimensionamiento. La copia deberá realizarse explícitamente, elemento por elemento.
- La representación interna forma parte del problema. Una solución funcional pero con complejidades incorrectas no obtiene el puntaje completo.
- Cada afirmación de complejidad deberá justificarse.
- Calificación.** Cada pregunta se califica sobre 1.0 con base en el código y en `analysis.pdf`. La sustentación **valida** esa nota: la calificación de cada pregunta se multiplica por un factor $f \in [0, 1]$ asignado según la capacidad del estudiante para explicar y modificar su propia solución. Una solución que el estudiante no pueda explicar recibe $f = 0$ en la pregunta correspondiente.
- Durante la sustentación el docente podrá modificar parámetros, solicitar una ejecución particular o preguntar por casos extremos.

Contexto general: The Snake Challenge

[0.0 pts]

Considere una versión simplificada del juego *Snake*. La serpiente está formada por una secuencia ordenada de posiciones

$$p_0, p_1, p_2, \dots, p_{k-1}, \quad p_0 = \text{TAIL}, \quad p_{k-1} = \text{HEAD},$$

donde cada posición se representa mediante:

```
public record Position(int row, int column) {}
```

Convención para todo el quiz: el índice lógico 0 corresponde siempre a la cola y el índice lógico $k - 1$ a la cabeza. La estructura debe preservar en todo momento este orden lógico.

Inicialmente:

$$\text{size} = 2, \quad \text{capacity} = 2, \quad \text{newCapacity} = 2 \cdot \text{capacity}.$$

Cuando la serpiente come una manzana, su longitud aumenta en uno.

Cuando realiza un movimiento sin comer, aparece una nueva cabeza y desaparece la cola anterior.

El siguiente pseudocódigo describe el comportamiento lógico del juego:

```
MOVE(direction)

    newHead <- computeNewHead(direction)

    if collision(newHead)           // current tail is occupied
        return GAME_OVER

    addHead(newHead)               // always executed first

    if foodAt(newHead)
        generateFood()
    else
        removeTail()
```

Precisiones.

- `collision` considera ocupadas todas las celdas actuales de la serpiente, incluida la cola, aunque vaya a liberarse durante ese mismo movimiento.
- El orden `addHead` $\rightarrow$ `removeTail` es obligatorio.
- En consecuencia, durante un movimiento sin comer el tamaño pasa **transitoriamente** de k a $k + 1$.

- Este crecimiento transitorio deberá tenerse en cuenta al analizar redimensionamientos y capacidad.

Importante: el pseudocódigo anterior especifica el **comportamiento lógico** del juego, pero no determina cuál debe ser la representación física utilizada para almacenar la serpiente.

Elegir y justificar dicha representación forma parte del problema.

1. Dynamic Array from Scratch

[1.0 pts]

Implemente desde cero una estructura genérica:

```
public class DynamicArray<T> {

    private T[] data;
    private int size;
    private int capacity;

    public DynamicArray(int initialCapacity) { ... }

    public int size() { ... }

    public int capacity() { ... }

    public T get(int index) { ... }

    public void set(int index, T value) { ... }

    public void append(T value) { ... }

    public T removeLast() { ... }

    private void resize(int newCapacity) { ... }
}
```

Su implementación deberá satisfacer:

$$0 \leq \text{size} \leq \text{capacity}, \quad \text{capacity} \geq 1.$$

El constructor deberá lanzar `IllegalArgumentException` si

$$\text{initialCapacity} < 1.$$

Los accesos con índices fuera de

$$[0, \text{size})$$

deberán lanzar `IndexOutOfBoundsException`, y `removeLast` sobre una estructura vacía deberá lanzar una excepción apropiada.

`removeLast` **no** reduce la capacidad.

Cuando

$$\text{size} = \text{capacity},$$

la siguiente operación `append` deberá duplicar la capacidad.

- Implemente completamente la clase.
- Escriba al menos **tres invariantes** que deben mantenerse antes y después de cada operación pública.
- Determine la complejidad **worst-case** de `get`, `set`, `append` y `removeLast`.
- Explique por qué el costo **worst-case** de `append` no contradice que posteriormente pueda obtenerse un costo amortizado constante.

2. Snake: una implementación aparentemente eficiente

[1.0 pts]

Un estudiante propone almacenar la serpiente en un `DynamicArray` de la siguiente manera:

$$\boxed{p_0} \boxed{p_1} \boxed{p_2} \boxed{\dots} \boxed{p_{k-1}} \quad p_0 = \text{TAIL}, \quad p_{k-1} = \text{HEAD}.$$

Para agregar una nueva cabeza utiliza:

```
snake.append(newHead);
```

y para eliminar la cola propone:

```
REMOVE_TAIL()

for i <- 1 to size - 1
    data[i - 1] <- data[i]
```

```
size <- size - 1
```

Considere una serpiente de longitud k que realiza m movimientos sin comer.

- Determine la complejidad temporal de una llamada a `REMOVE_TAIL()`.
- Determine la complejidad total de los m movimientos en función de m y k , suponiendo que la longitud permanece constante.
- Un estudiante afirma:

“Como `append()` es $O(1)$ amortizado, entonces `MOVE()` también es $O(1)$ amortizado.”

Determine si la afirmación es válida y justifique formalmente.

- Identifique el **bottleneck** de esta representación.

3. Redesign Challenge

[1.0 pts]

La implementación anterior no satisface el objetivo de realizar movimientos eficientes cuando la longitud de la serpiente es grande.

Rediseñe la representación interna utilizada por **Snake** de manera que se satisfagan simultáneamente:

`addHead()` = $O(1)$ amortizado,

`removeTail()` = $O(1)$ worst-case,

utilizando únicamente:

- un arreglo nativo de Java;
- un número constante de variables auxiliares.

No puede utilizar otra estructura de datos dinámica.

La clase **Snake** deberá gestionar directamente su propio arreglo y no podrá delegar su almacenamiento en el `DynamicArray` desarrollado en la Pregunta 1.

Puede reutilizar conceptos y técnicas desarrolladas previamente, pero **la representación física utilizada forma parte de la solución.**

Su diseño deberá funcionar correctamente incluso cuando los elementos que forman la serpiente **no ocupen posiciones físicas consecutivas de izquierda a derecha dentro del arreglo.**

- Defina las variables de estado adicionales que necesita y explique el significado de cada una.
- Defina formalmente cómo se transforma un índice lógico i , donde

$$0 \leq i < \text{size},$$

en la posición física correspondiente dentro del arreglo.

- Implemente en Java:

```
public void addHead(Position p)

public Position removeTail()

public Position get(int logicalIndex)
```

- Implemente `resize()` para su representación.
Después de un **resize**, los elementos deberán preservar exactamente el mismo orden lógico que tenían antes de la operación, independientemente de su distribución física anterior.
- Indique los invariantes que permiten demostrar que su representación y las operaciones anteriores son correctas.
- Traza.**
Partiendo del estado inicial

`size = capacity = 2,`

con cola A y cabeza B , ejecute:

N_1 (no come),

N_2 (come),

N_3 (no come),

N_4 (no come),

N_5 (come),

N_6 (no come).

Después de cada movimiento muestre:

- el contenido **físico completo** del arreglo, incluyendo las celdas vacías;
- el valor de todas las variables de estado utilizadas por su representación;
- `size`;
- `capacity`;
- si ocurrió o no un `resize`.

Restricción adicional:

Durante un movimiento ordinario, la solución no puede desplazar sistemáticamente los k elementos de la serpiente.

4. Amortized Analysis: demostrar, no afirmar

[1.0 pts]

Considere ahora una ejecución de m movimientos sobre la representación de la Pregunta 3.

La serpiente **no necesariamente crece en cada movimiento**.

Sea

$$g \leq m$$

el número total de manzanas consumidas durante la ejecución.

La longitud inicial es 2, la capacidad inicial es 2 y cada vez que el arreglo se llena se utiliza:

$$C_{\text{new}} = 2C_{\text{old}}.$$

Suponga el siguiente modelo de costo:

- escribir una posición nueva cuesta 1;
 - copiar una posición durante `resize` cuesta 1;
 - las operaciones aritméticas y actualización de índices cuestan $O(1)$.
- a) Utilizando el **Aggregate Method**, determine una cota asintótica, **en función de g** , para el número total de elementos copiados durante todos los `resize`.
Determine primero la longitud máxima que puede alcanzar transitoriamente la estructura, teniendo en cuenta el orden

`addHead` $\rightarrow$ `removeTail`.

- b) Demuestre que una secuencia de crecimientos tiene costo total lineal respecto al número de elementos almacenados.
No se acepta como demostración escribir únicamente

$$1 + 2 + 4 + \dots = O(n).$$

Deberá explicar por qué los términos de la suma corresponden a las capacidades existentes exactamente cuando ocurre cada `resize`.

- c) Determine el costo total de los m movimientos y el costo amortizado de `addHead()`.
d) Explique por qué m y g cumplen papeles diferentes en este análisis.
En particular, identifique qué parte del costo total depende del número de movimientos y qué parte depende del crecimiento de la serpiente.
e) Repita el argumento utilizando **uno** de los siguientes métodos:

Banker's Method o **Potential Method**.

Deberá definir explícitamente los créditos asignados o la función potencial utilizada.

Su argumento deberá tener en cuenta que `removeTail` elimina elementos: justifique por qué los créditos disponibles o el potencial siguen siendo suficientes para pagar cada futuro `resize`.

5. The Adversarial Snake

[1.0 pts]

Un adversario controla la aparición de las manzanas y puede hacer que la serpiente crezca en cualquier subconjunto de los movimientos.

Se consideran las siguientes políticas de crecimiento, aplicadas cuando el arreglo está lleno y debe insertarse un nuevo elemento:

Política	Nueva capacidad
A	$C + 1$
B	$2C$
C	$\lceil 1.5C \rceil$

Suponga que eventualmente la estructura alcanza una longitud máxima n , incluyendo los estados transitorios producidos por `addHead()` antes de `removeTail()`.

- Demuestre que, en ausencia de contracción, la elección de **cuándo** crece la serpiente no afecta el número total de elementos copiados hasta alcanzar longitud n .
¿Qué capacidad adicional tendría que poseer el adversario para poder aumentar el número de copias?
- Para cada política determine el número asintótico de operaciones de **resize**.
- Para cada política determine el número total de elementos copiados hasta alcanzar longitud n .
- Para cada política determine el costo amortizado **por operación de crecimiento**.
- Para cada política determine el espacio reservado que puede quedar sin utilizar inmediatamente después de un **resize**:
 - en valor absoluto;
 - como fracción de la capacidad.
- Discuta el compromiso (*trade-off*) entre:
 - tiempo de ejecución;
 - frecuencia de redimensionamiento;
 - número de elementos copiados;
 - memoria reservada.

Complete y **justifique** la siguiente tabla:

Política	# Resizes	Total copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
$C + 1$				
$2C$				
$\lceil 1.5C \rceil$				

Finalmente responda:

¿Es necesario duplicar la capacidad para obtener costo amortizado constante, o existe una familia más general de políticas

$$C_{\text{new}} = \lceil \alpha C \rceil$$

que conserva esta propiedad?

¿Para qué valores de α ?

Expresé el número total de elementos copiados en función de α y n .

Su respuesta deberá estar acompañada de un argumento matemático.

Sustentación oral **[factor $f \in [0, 1]$ por pregunta]**

La sustentación tendrá una duración máxima de **12 minutos**.

Tiempo	Actividad
0–3 min	Demostración de la implementación y ejecución de casos de prueba.
3–6 min	Explicación de la representación interna, invariantes, relación entre posiciones lógicas y físicas y procedimiento de resize .
6–9 min	Presentación del análisis amortizado y comparación de políticas de crecimiento.
9–12 min	Preguntas del docente y modificación de condiciones.

Durante la sustentación podrán formularse preguntas no conocidas previamente, por ejemplo:

- ¿Qué ocurre si el factor de crecimiento cambia de 2 a 1.25?
- ¿Qué pasaría si se utilizara

$$\lceil 1.25C \rceil$$

en lugar de

$$\lceil 1.25C \rceil$$

cuando $C = 2$?

- ¿Qué ocurre si se reduce la capacidad inmediatamente después de eliminar un elemento?
- Construya una secuencia de operaciones que provoque redimensionamientos repetidos bajo una política ingenua de crecimiento y contracción.
Por ejemplo: duplicar al llenarse y reducir a la mitad cuando

$$\text{size} \leq \frac{\text{capacity}}{2}.$$

- Si conoce únicamente **size** y **capacity**, ¿puede reconstruir la representación física de la serpiente? Justifique.
- ¿Por qué `new T[n]` no compila en Java? ¿Qué riesgo introduce el cast

```
(T[]) new Object[n]
```

?

- ¿Por qué el primer movimiento sin comer puede provocar un **resize** bajo las condiciones iniciales del problema?
- ¿Cuál es la diferencia entre el costo **worst-case** y el costo **amortizado** de una operación individual?
- Si su representación utiliza una función que relaciona posiciones lógicas y físicas, demuestre que dicha función siempre produce un índice válido del arreglo.

Formato de entrega

[0.0 pts]

La entrega deberá contener:

```
src/
  DynamicArray.java    // Pregunta 1
  Snake.java           // Pregunta 3
  Position.java
  Main.java            // reproduce la traza de P3

README.md
analysis.pdf
```

El archivo **README.md** deberá contener únicamente la versión de Java utilizada y las instrucciones necesarias para compilar y ejecutar la solución.

El documento **analysis.pdf** tendrá una extensión máxima de **cuatro páginas** y deberá contener:

- representación interna;
- invariantes;
- análisis de complejidad;
- demostración del análisis amortizado;
- comparación de las políticas de crecimiento;
- tabla correspondiente a la Pregunta 5.

El código entregado deberá ser exactamente el código utilizado durante la sustentación.

Nombre: _____

Documento: _____

	P1	P2	P3	P4	P5	Total
Nota escrita:	____ / 1.0	____ / 1.0	____ / 1.0	____ / 1.0	____ / 1.0	
Factor <i>f</i> :	____	____	____	____	____	
Nota final:	____	____	____	____	____	____ / 5.0